Università degli Studi di Napoli Federico II

Documentazione per Progetto

Basi di Dati

CdL Triennale in Informatica

Corso Basi di Dati

Simone Mollo
N86004951

Francesco Mistero
N86005098

Anno Accademico: 2025/2026

Indice

1 Introduzione **3**
1.1 Descrizione del Problema . . . 3

2 Progettazione Concettuale **4**
2.1 Class Diagram . . . 4
2.2 Ristrutturazione del Class Diagram . . . 5
2.2.1 Analisi delle chiavi primarie . . . 5
2.2.2 Analisi delle chiavi esterne (FK) . . . 5
2.2.3 Analisi delle ridondanze . . . 5
2.2.4 Analisi degli attributi strutturati (Enumerazioni) . . . 6
2.2.5 Analisi delle associazioni . . . 6
2.2.6 Analisi degli attributi derivati . . . 6
2.2.7 Analisi degli attributi a valore multiplo . . . 6
2.2.8 Analisi delle gerarchie di specializzazione . . . 6
2.3 Class Diagram Ristrutturato . . . 7
2.4 Dizionario delle Classi . . . 8
2.5 Dizionario delle Associazioni . . . 9
2.6 Dizionario dei Vincoli . . . 9

3 Progettazione Logica **10**
3.1 Schema Logico . . . 10

4 Progettazione Fisica **11**
4.1 Definizione delle Tabelle . . . 11
4.1.1 Definizione della tabella UTENTE . . . 11
4.1.2 Definizione della tabella BACHECA . . . 11
4.1.3 Definizione della tabella TODO . . . 11
4.1.4 Definizione della tabella CONDIVISIONE . . . 12
4.2 Vincoli e Trigger . . . 12
4.2.1 Creazione automatica delle bacheche . . . 12
4.2.2 Prevenzione dell'auto-condivisione . . . 13
4.2.3 Vincolo temporale sulla data di scadenza (INSERT) . . . 14
4.2.4 Vincoli di dominio e integrità dei dati . . . 15
4.3 Funzioni e Procedure . . . 16
4.3.1 Recupero dei ToDo scaduti . . . 16
4.3.2 Recupero dei ToDo completati . . . 17
4.3.3 Ricerca dei ToDo per limite di scadenza . . . 18
4.3.4 Ricerca testuale dei ToDo per titolo . . . 19
4.3.5 Creazione di una nuova attività (ToDo) . . . 20
4.3.6 Aggiornamento di un ToDo esistente . . . 21
4.3.7 Eliminazione di un ToDo . . . 22
4.3.8 Condivisione di un ToDo con un altro utente . . . 23
4.3.9 Revoca della condivisione . . . 24
4.3.10 Registrazione di un nuovo utente . . . 24
4.3.11 Eliminazione di un utente . . . 24
4.3.12 Spostamento di un ToDo tra bacheche . . . 25
4.3.13 Cambio posizione di un ToDo . . . 26

4.3.14 Gestione delle bacheche (CRUD) 27

1 Introduzione

Il seguente elaborato ha lo scopo di documentare la progettazione e lo sviluppo di una base di dati relazionale su DBMS PostgreSQL. Il progetto nasce come attività valutativa per il corso di Basi di Dati e riguarda la realizzazione di un sistema informativo per la gestione e l'organizzazione di attività personali ("Gestore ToDo"), ispirato alle funzionalità del software Trello.

L'idea di fondo è fornire un supporto dati stabile ad un'applicazione Java con GUI (Swing), capace di gestire utenti, attività e condivisioni, mantenendo coerenza e integrità delle informazioni anche durante operazioni tipiche come spostamenti tra bacheche, riordinamenti e filtri per scadenza.

1.1 Descrizione del Problema

La progettazione prevede lo sviluppo di una base di dati relazionale a supporto di un sistema per la gestione delle attività personali (ToDo Manager). Il sistema memorizza e organizza le informazioni relative agli utenti e alle loro credenziali, strutturando le attività in tre bacheche tematiche predefinite: *Università*, *Lavoro* e *Tempo Libero*.

Per ogni attività (ToDo), il database gestisce dettagli opzionali quali titolo, data di scadenza, descrizione testuale, collegamenti esterni (URL), immagine associata e parametri visivi come il colore di sfondo. Inoltre, viene gestita la logica di condivisione tramite una tabella associativa che traccia i destinatari e, opzionalmente, la data di condivisione.

La base di dati consente operazioni di inserimento, aggiornamento, eliminazione e ricerca, facilitando il monitoraggio delle scadenze, l'ordinamento personalizzato delle attività e la collaborazione tra utenti.

2 Progettazione Concettuale

In questo capitolo documentiamo la progettazione del database al suo livello di astrazione più alto. A partire dai requisiti, sono state individuate le entità principali del dominio e le relazioni che intercorrono tra di esse, ottenendo uno schema concettuale rappresentato mediante un Class Diagram UML. Lo scopo di questa fase è descrivere "cosa" rappresenta il sistema, senza ancora vincolarsi a scelte implementative tipiche del modello relazionale (chiavi surrogate, vincoli SQL, ecc.).

2.1 Class Diagram

Il Class Diagram seguente rappresenta la struttura concettuale del sistema. Le classi principali individuate sono:

- **Utente**: caratterizzato da username e password.
- **Bacheca**: contenitore delle attività, caratterizzato da descrizione e tipo (definito tramite enumerazione).
- **ToDo**: attività, con attributi opzionali come dataScadenza, url, colore, immagine e attributi gestionali come ordine e stato.
- **Condivisione**: entità associativa che traccia la condivisione tra Utente e ToDo, con eventuale dataCondivisione.

Sono presenti due enumerazioni:

- **TipoBacheca**: UNIVERSITA, LAVORO, TEMPO_LIBERO.
- **StatusTodo**: COMPLETATO, NON_COMPLETATO.

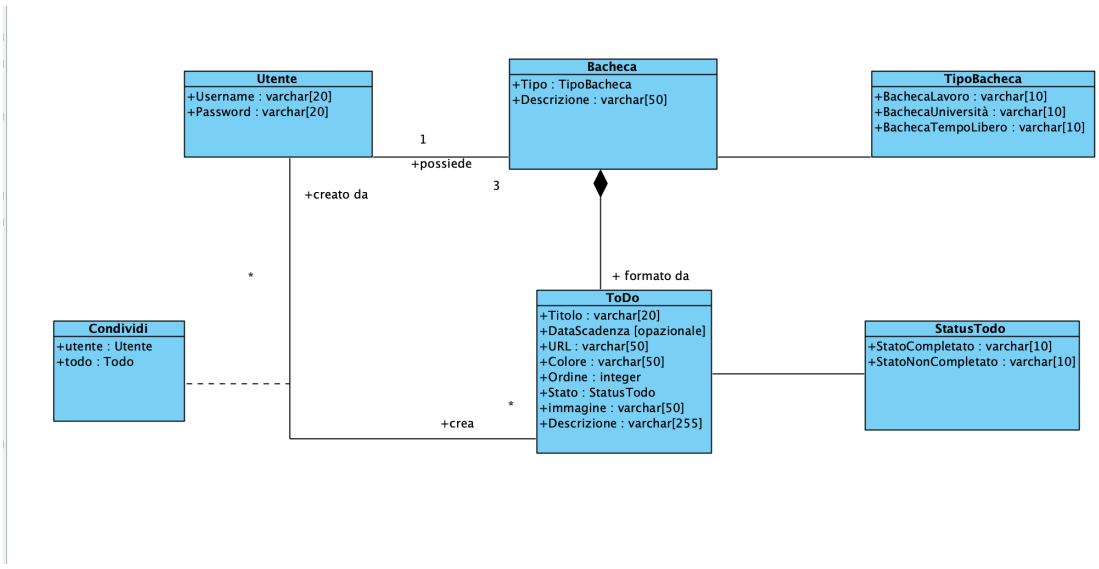

Figura 1: Class Diagram Concettuale

2.2 Ristrutturazione del Class Diagram

Si procede alla ristrutturazione del Class Diagram per renderlo idoneo alla traduzione in schema relazionale. In particolare, questa fase ha due obiettivi principali: (i) rendere espliciti gli identificatori delle entità (chiavi) e la modalità con cui le relazioni verranno implementate nel modello relazionale (chiavi esterne), e (ii) ridurre ridondanze e possibili incoerenze, mantenendo lo schema quanto più semplice e consistente possibile.

2.2.1 Analisi delle chiavi primarie

Per l'entità **Utente** si mantiene come identificatore la chiave naturale `username`, in quanto univoca e direttamente richiesta dalla traccia come credenziale di login. Questa scelta semplifica la progettazione e rende immediato il riferimento all'utente nelle altre tabelle.

Per le entità **Bacheca** e **ToDo** vengono invece introdotte chiavi surrogate di tipo intero autoincrementale (`id`). La motivazione è duplice: da un lato si garantisce un identificatore stabile e compatto, dall'altro si semplifica la gestione di operazioni frequenti come spostamenti, join e cancellazioni, evitando di basarsi su attributi testuali o modificabili.

Infine, **Condivisione** è una tabella associativa: l'identificazione naturale del singolo evento di condivisione è data dalla coppia delle chiavi esterne (`todo_id`, `username`), che diventa quindi chiave primaria composta. In questo modo si impedisce di inserire più volte la stessa condivisione dello stesso ToDo verso lo stesso utente.

2.2.2 Analisi delle chiavi esterne (FK)

Le relazioni del modello concettuale vengono tradotte nel modello relazionale tramite chiavi esterne:

- **Bacheca** contiene una chiave esterna `proprietario` verso `Utente(username)` per indicare l'utente proprietario della bacheca.
- **ToDo** contiene una chiave esterna `bacheca_id` verso `Bacheca(id)` per collegare ogni attività alla bacheca di appartenenza.
- **Condivisione** contiene due chiavi esterne: `todo_id` verso `ToDo(id)` e `username` verso `Utente(username)`.

Nella fase fisica, tali riferimenti vengono dotati di `ON DELETE CASCADE` per evitare record orfani: eliminando un utente si eliminano automaticamente le bacheche e le condivisioni collegate, ed eliminando un ToDo si eliminano automaticamente le sue condivisioni.

2.2.3 Analisi delle ridondanze

Non viene memorizzato esplicitamente un attributo "creatore" all'interno della tabella **ToDo**. La motivazione è evitare ridondanza e possibili incoerenze: il creatore coincide infatti con il proprietario della bacheca che contiene il ToDo, ed è sempre ricavabile tramite la join `ToDo` → `Bacheca` → `Utente`. In questo modo si mantiene un'unica fonte di verità e si riduce il rischio di dati contraddittori nel tempo.

2.2.4 Analisi degli attributi strutturati (Enumerazioni)

Le enumerazioni del modello concettuale vengono implementate fisicamente tramite tipi enumerativi nativi di PostgreSQL (`ENUM`). In particolare:

- `tipo_bacheca` $\in$ {UNIVERSITA, LAVORO, TEMPO_LIBERO}
- `status_todo` $\in$ {COMPLETATO, NON_COMPLETATO}

Questa scelta consente di vincolare direttamente i valori ammessi a livello di DBMS, riducendo la possibilità di inserimenti non validi e semplificando la logica lato applicazione.

2.2.5 Analisi delle associazioni

La relazione **Possiede** tra Utente e Bacheca nasce dal requisito delle tre bacheche standard. Operativamente, il sistema inizializza ogni utente con tre bacheche (una per ciascun tipo) tramite un trigger automatico in fase di registrazione. Tuttavia, essendo consentita l'eliminazione delle bacheche, nel tempo un utente può trovarsi con **0..3** bacheche standard: per questo motivo si distingue tra inizializzazione automatica e cardinalità effettiva.

La relazione **Contiene** (Bacheca-ToDo) è una relazione 1 a molti: una bacheca può contenere più ToDo, mentre un ToDo appartiene ad una sola bacheca.

La relazione **Condivisione** (ToDo-Utente) è una relazione molti-a-molti, implementata tramite tabella associativa. Questa scelta permette di rappresentare in modo naturale la visibilità dei ToDo condivisi e di gestire in modo semplice operazioni di inserimento e revoca della condivisione.

2.2.6 Analisi degli attributi derivati

Nel modello non sono stati individuati attributi derivati da memorizzare in modo persistente. Eventuali informazioni calcolabili (ad esempio la condizione di "ToDo scaduto") vengono ottenute a query time confrontando `data_scadenza` con `CURRENT_DATE`, evitando ridondanza e possibili inconsistenze dovute al trascorrere del tempo.

2.2.7 Analisi degli attributi a valore multiplo

Nel modello concettuale non sono presenti attributi a valore multiplo all'interno delle entità principali. L'unico caso assimilabile a una pluralità di valori è l'insieme di utenti con cui un ToDo è condiviso: tale informazione è stata correttamente modellata tramite l'entità associativa `Condivisione`, che risolve la relazione molti-a-molti tra `ToDo` e `Utente` e consente di gestire in modo naturale inserimento e revoca delle condivisioni.

2.2.8 Analisi delle gerarchie di specializzazione

Nel dominio applicativo non sono state individuate gerarchie di specializzazione tra entità. Gli utenti del sistema sono gestiti in modo uniforme e non sono previsti ruoli differenti con attributi aggiuntivi o vincoli strutturali diversi; pertanto il modello non richiede una scomposizione in sottoclassi.

2.3 Class Diagram Ristrutturato

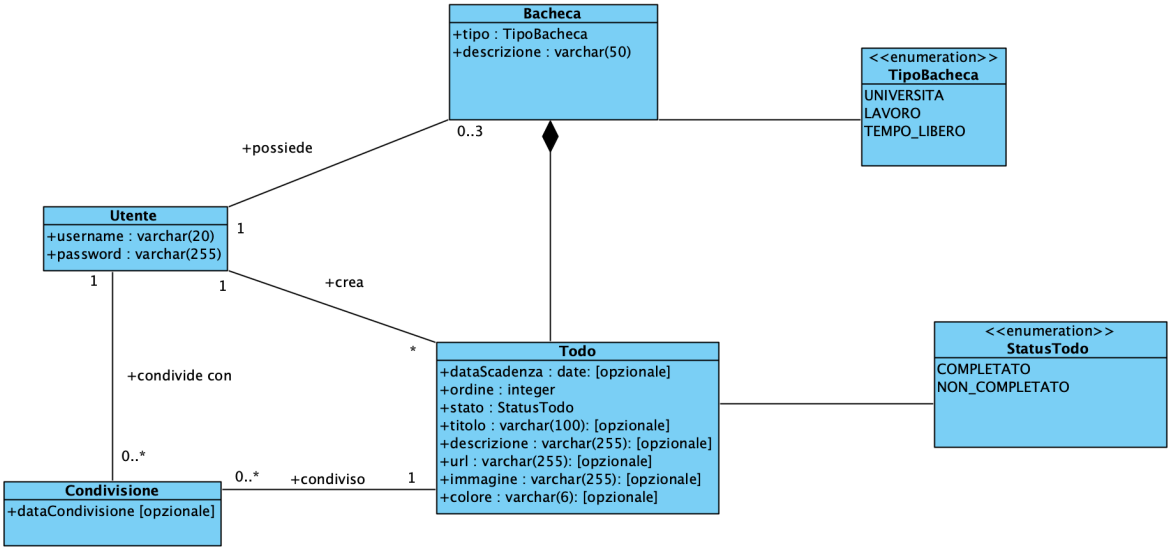

Figura 2: Class Diagram Ristrutturato con chiavi esplicite

2.4 Dizionario delle Classi

Classe	Descrizione	Attributi
Utente	Utente del sistema.	– <u>username</u> (varchar 20): PK. – password (varchar 255): NOT NULL.
Bacheca	Contenitore delle attività.	– <u>id</u> (int): PK. – tipo (tipo_bacheca): Enum PostgreSQL. – descrizione (varchar 50). – proprietario (FK → Utente.username). – UNIQUE: (proprietario, tipo).
ToDo	Attività da svolgere.	– <u>id</u> (int): PK. – bacheca_id (FK → Bacheca.id). – ordine (integer): NOT NULL. – stato (status_todo): default NON_COMPLETATO. – titolo (varchar 100): opzionale. – data_scadenza (date): opzionale. – url (varchar 255): opzionale. – descrizione (varchar 255): opzionale. – immagine (varchar 255): opzionale. – colore (varchar 6): opzionale. – UNIQUE: (bacheca_id, ordine).
Condivisione	Associa un ToDo ad un utente destinatario.	– <u>todo_id</u> (FK → ToDo.id). – <u>username</u> (FK → Utente.username). – dataCondivisione (date): opzionale. – PK composta: (todo_id, username).

2.5 Dizionario delle Associazioni

Nome	Descrizione	Classi Coinvolte
possiede	L'utente possiede fino a tre bacheche standard (una per ciascun tipo); la creazione iniziale è automatica.	Utente (1) $\leftrightarrow$ Bacheca (0..3)
formato da	Composizione: la bacheca contiene i ToDo.	Bacheca (1) $\leftrightarrow$ ToDo (0..*)
condivide con / condiviso	Gestita dalla tabella Condivisione: un ToDo può essere condiviso con uno o più utenti.	ToDo (1) $\leftarrow$ Condivisione $\rightarrow$ Utente (1)

2.6 Dizionario dei Vincoli

Vincolo	Descrizione
Valori Enum	tipo_bacheca $\in$ {UNIVERSITA, LAVORO, TEMPO_LIBERO}. status_todo $\in$ {COMPLETATO, NON_COMPLETATO}. Implementati tramite tipi ENUM di PostgreSQL.
Univocità Bacheca	UNIQUE(proprietario, tipo): un utente non può avere due bacheche dello stesso tipo.
Univocità Ordine ToDo	UNIQUE(bacheca_id, ordine): due ToDo non possono occupare la stessa posizione nella stessa bacheca.
Lunghezza Stringhe	Vincoli CHECK: titolo max 100 caratteri, descrizione max 255 caratteri.
Scadenza (business rule)	La data_scadenza, se presente, non può essere nel passato al momento dell'inserimento. Vincolo implementato tramite trigger BEFORE INSERT su todo.
Anti Auto-Condivisione	È vietato condividere un ToDo con il proprietario del ToDo stesso. Vincolo implementato tramite trigger BEFORE INSERT su condivisione.

3 Progettazione Logica

3.1 Schema Logico

Schema finale derivante dal diagramma ristrutturato e implementato in PostgreSQL.

- **Utente** (username, password)
- **Bacheca** (id, tipo, descrizione, proprietario)
  FK: proprietario → Utente(username) (ON DELETE CASCADE)
  UNIQUE: (proprietario, tipo)
- **ToDo** (id, bacheca_id, data_scadenza, ordine, stato, titolo, descrizione, url, immagine, colore)
  FK: bacheca_id → Bacheca(id) (ON DELETE CASCADE)
  UNIQUE: (bacheca_id, ordine)
- **Condivisione** (todo_id, username, dataCondivisione)
  FK: todo_id → ToDo(id) (ON DELETE CASCADE)
  FK: username → Utente(username) (ON DELETE CASCADE)
  PK composta: (todo_id, username)

4 Progettazione Fisica

In questo capitolo viene riportata l'implementazione fisica dello schema logico all'interno del DBMS PostgreSQL. Verranno presentate le definizioni delle tabelle e la creazione dei tipi enumerativi (ENUM) personalizzati, seguite dall'implementazione di trigger, funzioni e procedure.

4.1 Definizione delle Tabelle

Di seguito sono riportate le istruzioni SQL (DDL) per la creazione delle entità del database, complete dei vincoli di integrità.

4.1.1 Definizione della tabella UTENTE

La tabella Utente memorizza le credenziali di accesso al sistema. L'attributo username funge da chiave primaria.

```sql
CREATE TABLE utente (
  username VARCHAR(20) PRIMARY KEY,
  password VARCHAR(255) NOT NULL
);
```

Figura 3: Codice SQL per la creazione della tabella UTENTE

4.1.2 Definizione della tabella BACHECA

Viene definito un tipo enumerativo personalizzato tipo_bacheca. È presente un vincolo UNIQUE sulla coppia (proprietario, tipo).

```sql
CREATE TABLE bacheca (
  id SERIAL PRIMARY KEY,
  proprietario VARCHAR(20) NOT NULL REFERENCES utente(username) ON DELETE CASCADE,
  tipo tipo_bacheca NOT NULL,
  descrizione VARCHAR(50) NOT NULL DEFAULT '',
  UNIQUE (proprietario, tipo)
);
```

```sql
CREATE TYPE tipo_bacheca AS ENUM ('UNIVERSITA', 'LAVORO', 'TEMPO_LIBERO');
```

Figura 4: Codice SQL per la creazione della tabella BACHECA e del relativo tipo ENUM

4.1.3 Definizione della tabella TODO

Viene definito il tipo enumerativo status_todo. Un vincolo UNIQUE sulla coppia (bacheca_id, ordine) garantisce che due task non occupino la stessa posizione.

```sql
CREATE TABLE todo (
  id SERIAL PRIMARY KEY,
  bacheca_id INTEGER NOT NULL REFERENCES bacheca(id) ON DELETE CASCADE,
  data_scadenza DATE NULL,
  ordine INTEGER NOT NULL,
  stato status_todo NOT NULL DEFAULT 'NON_COMPLETATO',
  titolo VARCHAR(100) NULL,
  descrizione VARCHAR(255) NULL,
  url VARCHAR(255) NULL,
  immagine VARCHAR(255) NULL,
  colore VARCHAR(6) NULL,
  UNIQUE (bacheca_id, ordine)
);
```

```sql
CREATE TYPE status_todo  AS ENUM ('COMPLETATO', 'NON_COMPLETATO');
```

Figura 5: Codice SQL per la creazione della tabella TODO e del relativo tipo ENUM

4.1.4 Definizione della tabella CONDIVISIONE

Tabella associativa con PK composta e ON DELETE CASCADE su entrambe le FK.

```sql
CREATE TABLE condivisione (
  todo_id INTEGER NOT NULL REFERENCES todo(id) ON DELETE CASCADE,
  username VARCHAR(20) NOT NULL REFERENCES utente(username) ON DELETE CASCADE,
  dataCondivisione DATE NULL,
  PRIMARY KEY (todo_id, username)
);
```

Figura 6: Codice SQL per la creazione della tabella CONDIVISIONE

4.2 Vincoli e Trigger

In questa sezione vengono presentate le funzioni PL/pgSQL e i relativi trigger implementati per garantire il rispetto dei vincoli di business e per automatizzare alcune operazioni.

4.2.1 Creazione automatica delle bacheche

Per soddisfare il requisito delle tre bacheche standard, è stata implementata la funzione crea_bacheche() associata a un trigger AFTER INSERT su utente.

```sql
CREATE OR REPLACE FUNCTION crea_bacheche()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO bacheca (proprietario, tipo, descrizione)
  VALUES
    (NEW.username, 'UNIVERSITA', 'Universita'),
    (NEW.username, 'LAVORO', 'Lavoro'),
    (NEW.username, 'TEMPO_LIBERO', 'Tempo Libero');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_crea_bacheche
AFTER INSERT ON utente
FOR EACH ROW
EXECUTE FUNCTION crea_bacheche();
```

Figura 7: Codice SQL della funzione crea_bacheche()

4.2.2 Prevenzione dell'auto-condivisione

Per impedire un errore logico, viene bloccato l'inserimento in condivisione qualora il destinatario coincida con il proprietario del ToDo.

```sql
CREATE OR REPLACE FUNCTION check_no_self_share()
RETURNS TRIGGER AS $$
DECLARE
  owner VARCHAR(20);
BEGIN
  SELECT b.proprietario
  INTO owner
  FROM todo t
  JOIN bacheca b ON t.bacheca_id = b.id
  WHERE t.id = NEW.todo_id;

  IF owner IS NULL THEN
    RAISE EXCEPTION 'Todo % non trovato', NEW.todo_id;
  END IF;

  IF NEW.username = owner THEN
    RAISE EXCEPTION 'Non puoi condividere un ToDo con te stesso (%).', owner;
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_self_share
BEFORE INSERT ON condivisione
FOR EACH ROW
EXECUTE FUNCTION check_no_self_share();
```

Figura 8: Codice SQL della funzione check_no_self_share()

4.2.3 Vincolo temporale sulla data di scadenza (INSERT)

La regola secondo cui un ToDo non può essere inserito con scadenza nel passato è implementata tramite trigger BEFORE INSERT su todo. La scelta del trigger (e non di un CHECK con CURRENT_DATE) evita di bloccare la modifica di ToDo che diventano scaduti nel tempo.

```sql
CREATE OR REPLACE FUNCTION check_scadenza_insert()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.data_scadenza IS NOT NULL AND NEW.data_scadenza < CURRENT_DATE THEN
    RAISE EXCEPTION 'data_scadenza non puo essere nel passato';
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_scadenza_insert
BEFORE INSERT ON todo
FOR EACH ROW
EXECUTE FUNCTION check_scadenza_insert();
```

Figura 9: Codice SQL della funzione check_scadenza_insert()

4.2.4 Vincoli di dominio e integrità dei dati

Sono stati definiti vincoli CHECK sulla tabella todo per garantire limiti massimi di lunghezza su titolo e descrizione.

```sql
ALTER TABLE todo
  ADD CONSTRAINT check_lunghezza_titolo_todo
  CHECK (titolo IS NULL OR length(titolo) <= 100);

ALTER TABLE todo
  ADD CONSTRAINT check_lunghezza_descrizione_todo
  CHECK (descrizione IS NULL OR length(descrizione) <= 255);
```

Figura 10: Codice SQL per l'aggiunta dei vincoli CHECK sulla tabella TODO

4.3 Funzioni e Procedure

In questa sezione vengono illustrate funzioni e procedure PL/pgSQL implementate per incapsulare la logica lato server, supportando filtri, ricerche e operazioni CRUD.

4.3.1 Recupero dei ToDo scaduti

```sql
CREATE OR REPLACE FUNCTION get_todo_scaduti(p_username VARCHAR(20))
RETURNS TABLE (
  todo_id INTEGER,
  titolo VARCHAR(100),
  data_scadenza DATE,
  tipo_bacheca tipo_bacheca,
  ordine INTEGER,
  stato status_todo
) AS $$
BEGIN
  RETURN QUERY
  SELECT t.id, t.titolo, t.data_scadenza, b.tipo, t.ordine, t.stato
  FROM todo t
  JOIN bacheca b ON b.id = t.bacheca_id
  LEFT JOIN condivisione c ON c.todo_id = t.id
  WHERE t.data_scadenza IS NOT NULL
    AND t.data_scadenza < CURRENT_DATE
    AND (b.proprietario = p_username OR c.username = p_username)
  ORDER BY t.data_scadenza, b.tipo, t.ordine;
END;
$$ LANGUAGE plpgsql;
```

Figura 11: Codice SQL della funzione get_todo_scaduti()

4.3.2 Recupero dei ToDo completati

```sql
CREATE OR REPLACE FUNCTION get_todo_completati(p_username VARCHAR(20))
RETURNS TABLE (
  todo_id INTEGER,
  titolo VARCHAR(100),
  data_scadenza DATE,
  tipo_bacheca tipo_bacheca,
  ordine INTEGER
) AS $$
BEGIN
  RETURN QUERY
  SELECT t.id, t.titolo, t.data_scadenza, b.tipo, t.ordine
  FROM todo t
  JOIN bacheca b ON b.id = t.bacheca_id
  LEFT JOIN condivisione c ON c.todo_id = t.id
  WHERE t.stato = 'COMPLETATO'
    AND (b.proprietario = p_username OR c.username = p_username)
  ORDER BY b.tipo, t.ordine;
END;
$$ LANGUAGE plpgsql;
```

Figura 12: Codice SQL della funzione get_todo_completati()

4.3.3 Ricerca dei ToDo per limite di scadenza

```sql
CREATE OR REPLACE FUNCTION trova_todo_per_scadenza(
  p_username VARCHAR(20),
  p_data_limite DATE
)
RETURNS TABLE (
  todo_id INTEGER,
  titolo VARCHAR(100),
  data_scadenza DATE,
  tipo_bacheca tipo_bacheca,
  ordine INTEGER,
  stato status_todo
) AS $$
BEGIN
  RETURN QUERY
  SELECT t.id, t.titolo, t.data_scadenza, b.tipo, t.ordine, t.stato
  FROM todo t
  JOIN bacheca b ON b.id = t.bacheca_id
  LEFT JOIN condivisione c ON c.todo_id = t.id
  WHERE t.data_scadenza IS NOT NULL
    AND t.data_scadenza <= p_data_limite
    AND (b.proprietario = p_username OR c.username = p_username)
  ORDER BY t.data_scadenza, b.tipo, t.ordine;
END;
$$ LANGUAGE plpgsql;
```

Figura 13: Codice SQL della funzione trova_todo_per_scadenza()

4.3.4 Ricerca testuale dei ToDo per titolo

```sql
CREATE OR REPLACE FUNCTION cerca_todo_per_titolo(
  p_username VARCHAR(20),
  p_query TEXT
)
RETURNS TABLE (
  todo_id INTEGER,
  titolo VARCHAR(100),
  data_scadenza DATE,
  tipo_bacheca tipo_bacheca,
  ordine INTEGER,
  stato status_todo
) AS $$
BEGIN
  RETURN QUERY
  SELECT t.id, t.titolo, t.data_scadenza, b.tipo, t.ordine, t.stato
  FROM todo t
  JOIN bacheca b ON b.id = t.bacheca_id
  LEFT JOIN condivisione c ON c.todo_id = t.id
  WHERE (b.proprietario = p_username OR c.username = p_username)
    AND t.titolo IS NOT NULL
    AND t.titolo ILIKE '%' || p_query || '%'
  ORDER BY b.tipo, t.ordine;
END;
$$ LANGUAGE plpgsql;
```

Figura 14: Codice SQL della funzione cerca_todo_per_titolo()

4.3.5 Creazione di una nuova attività (ToDo)

```sql
CREATE OR REPLACE PROCEDURE crea_todo(
  p_bacheca_id INTEGER,
  p_titolo VARCHAR(100),
  p_descrizione VARCHAR(255),
  p_data_scadenza DATE,
  p_url VARCHAR(255),
  p_immagine VARCHAR(255),
  p_colore VARCHAR(6)
)
LANGUAGE plpgsql AS $$
DECLARE
  new_ordine INTEGER;
BEGIN
  SELECT COALESCE(MAX(ordine), 0) + 1
  INTO new_ordine
  FROM todo
  WHERE bacheca_id = p_bacheca_id;

  INSERT INTO todo (bacheca_id, ordine, titolo, descrizione, data_scadenza, url, immagine, colore)
  VALUES (p_bacheca_id, new_ordine, p_titolo, p_descrizione, p_data_scadenza, p_url, p_immagine, p_colore);
END;
$$;
```

Figura 15: Codice SQL della procedura crea_todo()

4.3.6 Aggiornamento di un ToDo esistente

```
CREATE OR REPLACE PROCEDURE aggiorna_todo(
  p_todo_id INTEGER,
  p_titolo VARCHAR(100),
  p_descrizione VARCHAR(255),
  p_data_scadenza DATE,
  p_url VARCHAR(255),
  p_immagine VARCHAR(255),
  p_colore VARCHAR(6),
  p_stato status_todo
)
LANGUAGE plpgsql AS $$
BEGIN
  UPDATE todo
  SET titolo = p_titolo,
      descrizione = p_descrizione,
      data_scadenza = p_data_scadenza,
      url = p_url,
      immagine = p_immagine,
      colore = p_colore,
      stato = p_stato
  WHERE id = p_todo_id;
END;
$$;
```

Figura 16: Codice SQL della procedura aggiorna_todo()

4.3.7 Eliminazione di un ToDo

```sql
CREATE OR REPLACE PROCEDURE elimina_todo(p_todo_id INTEGER)
LANGUAGE plpgsql AS $$
BEGIN
  DELETE FROM todo WHERE id = p_todo_id;
END;
$$;

CREATE OR REPLACE PROCEDURE sposta_todo_bacheca(
  p_todo_id INTEGER,
  p_nuova_bacheca_id INTEGER
)
LANGUAGE plpgsql AS $$
DECLARE
  new_ordine INTEGER;
BEGIN
  SELECT COALESCE(MAX(ordine), 0) + 1
  INTO new_ordine
  FROM todo
  WHERE bacheca_id = p_nuova_bacheca_id;

  UPDATE todo
  SET bacheca_id = p_nuova_bacheca_id,
      ordine = new_ordine
  WHERE id = p_todo_id;
END;
$$;
```

Figura 17: Codice SQL della procedura elimina_todo()

4.3.8 Condivisione di un ToDo con un altro utente

```sql
CREATE OR REPLACE PROCEDURE condividi_todo(
  p_todo_id INTEGER,
  p_username VARCHAR(20),
  p_dataCondivisione DATE
)
LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO condivisione(todo_id, username, dataCondivisione)
  VALUES (p_todo_id, p_username, p_dataCondivisione);
END;
$$;

CREATE OR REPLACE PROCEDURE rimuovi_condivisione(
  p_todo_id INTEGER,
  p_username VARCHAR(20)
)
LANGUAGE plpgsql AS $$
BEGIN
  DELETE FROM condivisione
  WHERE todo_id = p_todo_id
    AND username = p_username;
END;
$$;
```

Figura 18: Codice SQL della procedura condividi_todo()

4.3.9 Revoca della condivisione

```
CREATE OR REPLACE PROCEDURE rimuovi_condivisione(
  p_todo_id INTEGER,
  p_username VARCHAR(20)
)
LANGUAGE plpgsql AS $$
BEGIN
  DELETE FROM condivisione
  WHERE todo_id = p_todo_id
    AND username = p_username;
END;
$$;
```

Figura 19: Codice SQL della procedura rimuovi_condivisione()

4.3.10 Registrazione di un nuovo utente

```
CREATE OR REPLACE PROCEDURE crea_utente(p_username VARCHAR(20), p_password VARCHAR(255))
LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO utente(username, password)
  VALUES (p_username, p_password);
END;
$$;
```

Figura 20: Codice SQL della procedura crea_utente()

4.3.11 Eliminazione di un utente

```
CREATE OR REPLACE PROCEDURE elimina_utente(p_username VARCHAR(20))
LANGUAGE plpgsql AS $$
BEGIN
  DELETE FROM utente WHERE username = p_username;
END;
$$;
```

Figura 21: Codice SQL della procedura elimina_utente()

4.3.12 Spostamento di un ToDo tra bacheche

```
CREATE OR REPLACE PROCEDURE sposta_todo_bacheca(
  p_todo_id INTEGER,
  p_nuova_bacheca_id INTEGER
)
LANGUAGE plpgsql AS $$
DECLARE
  new_ordine INTEGER;
BEGIN
  SELECT COALESCE(MAX(ordine), 0) + 1
  INTO new_ordine
  FROM todo
  WHERE bacheca_id = p_nuova_bacheca_id;

  UPDATE todo
  SET bacheca_id = p_nuova_bacheca_id,
      ordine = new_ordine
  WHERE id = p_todo_id;
END;
$$;
```

Figura 22: Codice SQL della procedura sposta_todo_bacheca()

4.3.13 Cambio posizione di un ToDo

```sql
CREATE OR REPLACE PROCEDURE cambia_posizione_todo(
  p_todo_id INTEGER,
  p_nuovo_ordine INTEGER
)
LANGUAGE plpgsql AS $$
DECLARE
  v_bacheca_id INTEGER;
  v_old_ordine INTEGER;
BEGIN
  SELECT bacheca_id, ordine
  INTO v_bacheca_id, v_old_ordine
  FROM todo
  WHERE id = p_todo_id;

  IF v_bacheca_id IS NULL THEN
    RAISE EXCEPTION 'Todo % non trovato', p_todo_id;
  END IF;

  IF p_nuovo_ordine < 1 THEN
    RAISE EXCEPTION 'Nuovo ordine deve essere >= 1';
  END IF;

  IF p_nuovo_ordine = v_old_ordine THEN
    RETURN;
  END IF;

  UPDATE todo SET ordine = -1 WHERE id = p_todo_id;

  IF p_nuovo_ordine < v_old_ordine THEN
    UPDATE todo
    SET ordine = ordine + 1
    WHERE bacheca_id = v_bacheca_id
      AND ordine >= p_nuovo_ordine
      AND ordine < v_old_ordine;
  ELSE
    UPDATE todo
    SET ordine = ordine - 1
    WHERE bacheca_id = v_bacheca_id
      AND ordine > v_old_ordine
      AND ordine <= p_nuovo_ordine;
  END IF;

  UPDATE todo SET ordine = p_nuovo_ordine WHERE id = p_todo_id;
END;
$$;
```

Figura 23: Codice SQL della procedura cambia_posizione_todo()

4.3.14 Gestione delle bacheche (CRUD)

```
CREATE OR REPLACE PROCEDURE crea_bacheca(
  p_proprietario VARCHAR(20),
  p_tipo tipo_bacheca,
  p_descrizione VARCHAR(50)
)
LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO bacheca(proprietario, tipo, descrizione)
  VALUES (p_proprietario, p_tipo, COALESCE(p_descrizione, ''));
END;
$$;
```

Figura 24: Codice SQL della procedura crea_bacheca()

```
CREATE OR REPLACE PROCEDURE aggiorna_bacheca(
  p_bacheca_id INTEGER,
  p_descrizione VARCHAR(50)
)
LANGUAGE plpgsql AS $$
BEGIN
  UPDATE bacheca
  SET descrizione = COALESCE(p_descrizione, '')
  WHERE id = p_bacheca_id;
END;
$$;
```

Figura 25: Codice SQL della procedura aggiorna_bacheca()

```
CREATE OR REPLACE PROCEDURE elimina_bacheca(p_bacheca_id INTEGER)
LANGUAGE plpgsql AS $$
BEGIN
  DELETE FROM bacheca WHERE id = p_bacheca_id;
END;
$$;
```

Figura 26: Codice SQL della procedura elimina_bacheca()